```
# ======================================================================
# TASK 1: DATA CLEANING, FEATURE ENCODING, & PREPARATION (UPDATED)
# ======================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, f1_score, classification_report

# 1. Load the airline passenger survey dataset using your new file name
file_name = 'ReadMe.csv'
df = pd.read_csv(file_name)

# 2. Handle missing data values (Impute Arrival Delay with its median)
if df['Arrival Delay in Minutes'].isnull().sum() > 0:
    arrival_median = df['Arrival Delay in Minutes'].median()
    df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(arri

# 3. Encode categorical features and target variable
df['satisfaction'] = df['satisfaction'].map({'satisfied': 1, 'dissatisfied': 0}

# Use pd.get_dummies to encode operational categories cleanly
categorical_cols = ['Customer Type', 'Type of Travel', 'Class']
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

print(f"🎉 Success! '{file_name}' loaded perfectly.")
print(f"Dataset dimensions: {df_encoded.shape}")
```

```
🎉 Success! 'ReadMe.csv' loaded perfectly.
Dataset dimensions: (129880, 23)
```

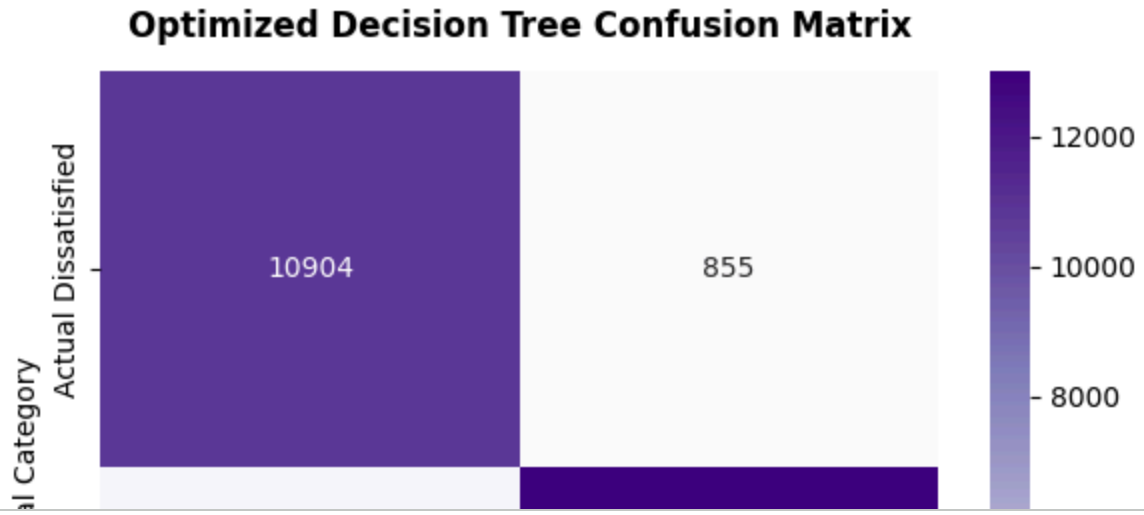
```
# ======================================================================
# TASK 2: STRATIFIED SPLITTING & GRIDSEARCHCV OPTIMIZATION
# ======================================================================

# Isolate features (X) and classification label (y)
X = df_encoded.drop(columns=['satisfaction'])
y = df_encoded['satisfaction']

# Unbiased 80/20 train-test split stratified on class balance
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Define search space parameters to prevent decision tree overfitting
param_grid = {
    'max_depth': [4, 6, 8, 10],
    'min_samples_split': [20, 50, 100],
    'min_samples_leaf': [10, 20, 50]
}

# Run Cross-Validated Grid Search maximizing F1-score for 'Satisfied' (Class 1)
dt_base = DecisionTreeClassifier(random_state=42)
```

```
grid_search = GridSearchCV(
    estimator=dt_base,
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
grid_search.fit(X_train, y_train)

# Store the absolute best estimator configuration
best_dt_model = grid_search.best_estimator_

print("--- 🌟 Optimal Hyperparameters Discovered ---")
print(grid_search.best_params_)
```

```
--- 🌟 Optimal Hyperparameters Discovered ---
{'max_depth': 10, 'min_samples_leaf': 10, 'min_samples_split': 20}
```

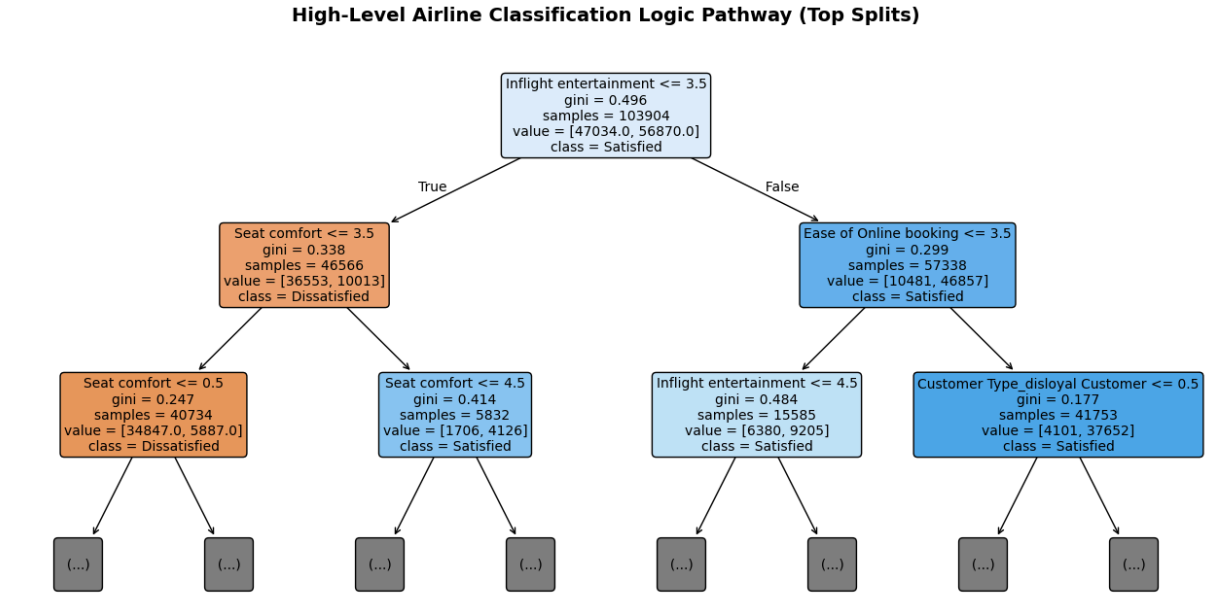
```
# ======================================================================
# TASK 3: EVALUATION METRICS FOR THE "SATISFIED" CLASS
# ======================================================================

# Generate predictions using optimized parameters
y_pred_dt = best_dt_model.predict(X_test)

# Extract core performance figures
dt_f1 = f1_score(y_test, y_pred_dt)
cm_dt = confusion_matrix(y_test, y_pred_dt)

print(f"Decision Tree F1-Score (Satisfied Class): {dt_f1:.4f}\n")
print("Detailed Classification Report:\n", classification_report(y_test, y_pred

# Visualize Confusion Matrix using Seaborn
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm_dt, annot=True, fmt='d', cmap='Purples',
    xticklabels=['Pred Dissatisfied', 'Pred Satisfied'],
    yticklabels=['Actual Dissatisfied', 'Actual Satisfied']
)
plt.title('Optimized Decision Tree Confusion Matrix', fontsize=12, fontweight=
plt.ylabel('Actual Category')
plt.xlabel('Predicted Category')
plt.tight_layout()
plt.show()
```

```
Decision Tree F1-Score (Satisfied Class): 0.9265

Detailed Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.93      0.91     11759
           1       0.94      0.91      0.93     14217

    accuracy                           0.92     25976
   macro avg       0.92      0.92      0.92     25976
weighted avg       0.92      0.92      0.92     25976
```




```
# ======================================================================
# TASK 4: PATHWAY VISUALIZATION & DRIVER FEATURE IMPORTANCE
# ======================================================================

# 1. Plot the high-level logic pathways (limited to depth=2 for presentation cl
plt.figure(figsize=(16, 8))
plot_tree(
    best_dt_model,
    max_depth=2,
    feature_names=X.columns,
    class_names=['Dissatisfied', 'Satisfied'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("High-Level Airline Classification Logic Pathway (Top Splits)", fonts
plt.show()

# 2. Extract and rank structural Feature Importance metrics
importances = best_dt_model.feature_importances_
feature_ranking = pd.DataFrame({
    'Operational Feature': X.columns,
    'Importance Weight': importances
}).sort_values(by='Importance Weight', ascending=False).reset_index(drop=True)

print("--- 🎯 Operational Feature Importance Rankings ---")
print(feature_ranking.head(10))

# Plot top drivers
```

```
plt.figure(figsize=(10, 5))
sns.barplot(data=feature_ranking.head(8), x='Importance Weight', y='Operational
plt.title('Top 8 Operational Drivers of Airline Passenger Satisfaction', fonts
plt.xlabel('Importance Weight')
plt.tight_layout()
plt.show()
```




```
--- 🎯 Operational Feature Importance Rankings ---
                  Operational Feature  Importance Weight
0             Inflight entertainment           0.492751
1                        Seat comfort           0.208535
2             Ease of Online booking           0.065152
3    Customer Type_disloyal Customer           0.032520
4  Departure/Arrival time convenient           0.026642
5     Type of Travel_Personal Travel           0.023441
6                     Flight Distance           0.019317
7                   Leg room service           0.015492
8                     Online support           0.014871
9                   On-board service           0.013801
/tmp/ipykernel_2581/2804373224.py:31: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remov
```

```
# ======================================================================
# TASK 5: LOGISTIC REGRESSION BASELINE BENCHMARK
# ======================================================================
from sklearn.preprocessing import StandardScaler

# Scale variables for stable Logistic Regression convergence
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(X_train_scaled, y_train)
y_pred_lr = log_reg.predict(X_test_scaled)
```

```
print(f"Logistic Regression F1-Score: {f1_score(y_test, y_pred_lr):.4f}")
print("\nLogistic Regression Matrix Breakdown:\n", confusion_matrix(y_test, y_
```

```
Logistic Regression F1-Score: 0.8436

Logistic Regression Matrix Breakdown:
 [[ 9546  2213]
 [ 2232 11985]]
```